

Heart Disease Prediction — MLOps End-to-End Pipeline

Assignment Report

Student	Mamatha Maganti (2025cs05051)
Course	MLOps — M.Tech Semester 2 (AMLCSZG523)
Date	May 2026
Repository	github.com/Mamatha-51/mlops-assignment

Table of Contents

1. Introduction
 2. Setup & Installation Instructions
 3. Data Acquisition & Exploratory Data Analysis
 4. Feature Engineering & Model Development
 5. Experiment Tracking with MLflow
 6. Model Packaging & Reproducibility
 7. CI/CD Pipeline & Automated Testing
 8. Model Containerization & API Design
 9. Production Deployment
 10. Monitoring & Logging
 11. System Architecture
 12. Conclusion & Future Work
 13. Evidence Logs
 14. References
-

1. Introduction

1.1 Problem Statement

Heart disease is one of the leading causes of death globally, accounting for approximately 17.9 million deaths annually (WHO). Early detection and risk assessment can significantly improve patient outcomes through timely intervention.

This project builds a complete end-to-end Machine Learning Operations (MLOps) pipeline to predict heart disease risk based on patient clinical data from the UCI Heart Disease dataset. The solution demonstrates the full ML lifecycle — from data acquisition through production monitoring — following industry-standard MLOps best practices.

1.2 Objectives

- Build and evaluate classification models for heart disease prediction
- Implement comprehensive experiment tracking for reproducibility
- Develop production-ready REST API with interactive UI
- Containerize and orchestrate using Docker and Kubernetes
- Establish monitoring and observability with Prometheus and Grafana
- Automate quality gates through CI/CD pipelines

1.3 Dataset Description

The UCI Heart Disease dataset contains **303 patient records** with **13 input features** and a binary target variable:

Category	Features
Demographic	age, sex
Clinical	chest pain type (cp), resting BP (trestbps), cholesterol (chol), fasting blood sugar (fbs), resting ECG (restecg)
Exercise	max heart rate (thalach), exercise angina (exang), ST depression (oldpeak), ST slope (slope)
Diagnostic	major vessels colored (ca), thalassemia (thal)
Target	0 = No Disease, 1 = Heart Disease

Source: UCI Machine Learning Repository

1.4 Technology Stack

Component	Technology	Purpose
Language	Python 3.11+	Core development
ML Framework	Scikit-learn	Model training & evaluation
API Framework	FastAPI + Flask	REST API + Browser UI
Experiment Tracking	MLflow	Parameter, metric & artifact logging
Testing	Pytest (24 tests)	Unit & integration testing
CI/CD	GitHub Actions	Automated pipeline (4 stages)
Containerization	Docker + Compose	Application packaging
Orchestration	Kubernetes (Minikube)	Production deployment
Monitoring	Prometheus + Grafana	Metrics & dashboards

2. Setup & Installation Instructions

2.1 Prerequisites

Requirement	Version
Python	3.10+ (tested on 3.11, 3.14)
Docker Desktop	Latest
minikube	v1.38+
kubectl	Compatible with cluster
Git	Latest

2.2 Local Setup (Step-by-Step)

Step	Command
1. Clone repository	<code>git clone https://github.com/Mamatha-51/mlops-assignment.git && cd mlops-assignment</code>
2. Create virtual environment	<code>python3 -m venv .venv && source .venv/bin/activate</code>
3. Install dependencies	<code>pip install -r requirements.txt</code>
4. Run EDA notebook	<code>jupyter notebook notebooks/EDA.ipynb</code>
5. Train models	<code>python src/models/train.py</code>
6. Run unit tests	<code>python -m pytest tests/ -v</code>
7. Start FastAPI server	<code>uvicorn src.api.model_app:app --reload</code>
8. Start Flask UI	<code>python src/api/app.py</code>
9. View MLflow dashboard	<code>mlflow ui --backend-store-uri mlruns/</code>

3. Data Acquisition & Exploratory Data Analysis

3.1 Data Acquisition

The dataset (`data/heart.csv`) was sourced from the UCI Machine Learning Repository. It is a well-established benchmark dataset for binary classification tasks in healthcare ML.

3.2 Data Quality Assessment

The preprocessing pipeline (`src/data/preprocess.py`) handles data quality:

Check	Result
Total records	303
Input features	13 (5 numerical + 8 categorical)
Target variable	Binary (0/1)

Check	Result
Missing values (after imputation)	0
Duplicate records	1 (removed)
Class balance	46% No Disease / 54% Disease

Imputation strategy: - Numerical columns → median imputation - Categorical columns → mode imputation

3.3 Exploratory Data Analysis

Comprehensive EDA was performed in notebooks/EDA.ipynb with 15+ visualizations:

Key Statistical Findings:

Feature	Insight
Age	Range 29–77, mean ~54 years, slightly right-skewed
Sex	68.3% male, 31.7% female
Chest Pain	Type 0 most common (47.2%); Type 2 has highest disease rate
Resting BP	Mostly 120–140 mmHg; mild positive skew
Cholesterol	Mean ~246 mg/dl; notable outliers above 400
Max Heart Rate	Strong negative correlation with heart disease
Oldpeak (ST depression)	Higher values strongly associated with disease
Exercise Angina	Patients with angina have 2.5× higher disease rate

Correlation Highlights: - cp (chest pain type) — strongest positive predictor - thalach (max heart rate) — strongest negative predictor - oldpeak, exang, ca — significant positive predictors - Low multicollinearity among features (VIF < 5 for all)

3.4 Generated Visualizations

Visualization	File
Feature Distributions (Histograms)	screenshots/feature_histograms.png
Box Plots (Outlier Detection)	screenshots/boxplots.png
Categorical Feature Analysis	screenshots/categorical_features.png

Visualization	File
Correlation Heatmap	screenshots/correlation_heatmap.png
Class Balance	screenshots/class_balance.png
Missing Value Heatmap	screenshots/missing_values_heatmap.png
Pair Plot (Key Features)	screenshots/pairplot.png
Age Group Analysis	screenshots/age_group_analysis.png
Violin Plots	screenshots/violin_plots.png

4. Feature Engineering & Model Development

4.1 Feature Engineering

A preprocessing pipeline was built using Scikit-learn's Pipeline with StandardScaler:

- All 13 features standardized to zero mean and unit variance
- Scaler fitted **only** on training data (preventing data leakage)
- Pipeline persisted as a single .pkl artifact for consistent train-serve behavior

4.2 Data Split Strategy

Parameter	Value
Train/Test ratio	80% / 20%
Training samples	242
Test samples	61
Stratification	Yes (maintains class balance)
Random state	42 (fixed for reproducibility)

4.3 Models Trained

Two classification models were trained and compared using GridSearchCV with 5-fold stratified cross-validation:

Model 1: Logistic Regression

Hyperparameter	Search Space
C (regularization)	{0.01, 0.1, 1, 10}
solver	{lbfgs, liblinear}

Hyperparameter	Search Space
penalty	l2

Rationale: Strong interpretable baseline; performs well on linearly separable data.

Model 2: Random Forest Classifier

Hyperparameter	Search Space
n_estimators	{100, 200}
max_depth	{5, 10, None}
min_samples_split	{2, 5}

Rationale: Ensemble method that captures non-linear interactions and provides feature importance.

4.4 Model Evaluation Results

Cross-Validation Results (5-Fold Stratified):

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8264 ± 0.028	0.8439 ± 0.033	0.7648 ± 0.080	0.7993 ± 0.043	0.9004 ± 0.016
Random Forest	0.8139 ± 0.014	0.8295 ± 0.044	0.7557 ± 0.076	0.7867 ± 0.027	0.8837 ± 0.031

Hold-out Test Set Results:

Model	ROC-AUC	Accuracy	F1-Score
Logistic Regression	0.9578	0.87	0.87
Random Forest	0.9551	0.90	0.90

Generated evaluation artifacts:

Artifact	File
Confusion Matrix — LR	screenshots/cm_logistic_regression.png
Confusion Matrix — RF	screenshots/cm_random_forest.png
ROC Curve — LR	screenshots/roc_logistic_regression.png
ROC Curve — RF	screenshots/roc_random_forest.png
Feature Importance	screenshots/feature_importance_*.png
Model Comparison Chart	screenshots/model_comparison.png

4.5 Best Model Selection

Criterion	Decision
Selected Model	Logistic Regression
Selection Metric	Highest mean CV ROC-AUC (0.9004)
Best Hyperparameters	C = 0.1, solver = liblinear
Rationale	Highest generalization performance measured by stratified 5-fold CV ROC-AUC; simpler model with better interpretability

Full training output: See screenshots/model_training.log

5. Experiment Tracking with MLflow

5.1 MLflow Integration

MLflow was integrated into the training pipeline (src/models/train.py) to provide comprehensive experiment tracking. All experiments are stored locally in the mlruns/ directory.

5.2 Items Tracked Per Run

Category	Items Logged
Parameters	model_type, all hyperparameters, dataset_hash, dataset_shape, train_size, test_size, feature names
Metrics	cv_accuracy_mean/std, cv_precision_mean/std, cv_recall_mean/std, cv_roc_auc_mean/std, cv_f1_mean/std, test_roc_auc, test_precision, test_recall, test_f1, training_time_seconds
Artifacts	Trained model (sklearn format), confusion matrix (PNG), ROC curve (PNG), feature importance plot (PNG), model comparison chart (PNG)

5.3 Accessing the Dashboard

```
mlflow ui --backend-store-uri mlruns/  
# Open http://127.0.0.1:5000
```

5.4 Value Delivered

Capability	Benefit
Run comparison	Side-by-side metric comparison

Capability	Benefit
Artifact versioning	across models Every model version with associated plots stored
Reproducibility	Full parameter + data hash logging enables exact replication
Audit trail	Complete history of all training decisions

6. Model Packaging & Reproducibility

6.1 Model Artifacts

The production model is packaged as a single sklearn Pipeline object:

```
src/models/heart_model.pkl
└─ Pipeline
    └─ StandardScaler (fitted on training data)
        └─ LogisticRegression (C=0.1, solver=liblinear)
```

6.2 Inference Pipeline

The HeartDiseasePredictor class (src/models/inference.py) provides: - Automatic model loading from .pkl artifact - Raw dictionary input → preprocessed → prediction in a single call - Returns: prediction (0/1), label, confidence score, and risk level - Supports both single and batch predictions

Verified inference output (from screenshots/Inference.log):

```
Input: {'age': 55, 'sex': 1, 'cp': 2, 'trestbps': 130, 'chol': 250,
        'fbs': 0, 'restecg': 1, 'thalach': 150, 'exang': 0,
        'oldpeak': 1.5, 'slope': 1, 'ca': 0, 'thal': 3}
```

Prediction: No Heart Disease

Confidence: 0.1651

Risk Level: Low Risk

6.3 Reproducibility Guarantees

Mechanism	Purpose
requirements.txt	Pinned package versions
Pipeline serialization	Scaler + model saved together
random_state=42	Fixed across all random operations
Stratified split	Consistent class distribution
Dataset hash (MLflow)	Detects any data changes

7. CI/CD Pipeline & Automated Testing

7.1 Unit Test Suite

24 tests across 3 test files ensure correctness at every layer:

tests/test_preprocess.py (8 tests): - Data loading returns valid DataFrame - No missing values post-cleaning - All expected columns present - Target is binary, feature count is 13 - Invalid path raises exception - Missing value handling works correctly

tests/test_model.py (11 tests): - Model registry has ≥ 2 entries with pipeline + param_grid - Cross-validation returns all expected metrics - Predictor loads successfully - Prediction returns dict with required keys - Prediction is binary (0 or 1) - Confidence in [0, 1] range - Risk level is valid category - Batch prediction works correctly

tests/test_api.py (5 tests): - Root endpoint returns 200 with API info - Health endpoint returns “healthy” status - Predict endpoint returns valid prediction - Missing fields return 422 validation error - Metrics endpoint is accessible

Execution Results (24/24 passed):

```
(.venv) mammagan@MAMMAGAN-M-TC73 MLOPs-Final % python -m pytest tests/ -v
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0
collected 24 items
```

```
tests/test_api.py::TestAPI::test_root PASSED [ 4%]
tests/test_api.py::TestAPI::test_health PASSED [ 8%]
tests/test_api.py::TestAPI::test_predict_valid PASSED [ 12%]
tests/test_api.py::TestAPI::test_predict_missing_field PASSED [ 16%]
tests/test_api.py::TestAPI::test_metrics_endpoint PASSED [ 20%]
tests/test_model.py::TestTraining::test_models_dict_has_entries PASSED [ 25%]
tests/test_model.py::TestTraining::test_each_model_has_pipeline_and_grid PASSED [ 29%]
tests/test_model.py::TestTraining::test_cross_val_evaluate_returns_metrics PASSED [ 33%]
tests/test_model.py::TestInference::test_predictor_loads PASSED [ 37%]
tests/test_model.py::TestInference::test_predict_returns_dict PASSED [ 41%]
tests/test_model.py::TestInference::test_predict_has_required_keys PASSED [ 45%]
tests/test_model.py::TestInference::test_prediction_is_binary PASSED [ 50%]
tests/test_model.py::TestInference::test_confidence_in_range PASSED [ 54%]
tests/test_model.py::TestInference::test_risk_level_valid PASSED [ 58%]
tests/test_model.py::TestInference::test_predict_batch PASSED [ 62%]
tests/test_model.py::TestInference::test_sample_input_has_all_features PASSED [ 66%]
tests/test_preprocess.py::TestLoadData::test_returns_dataframe PASSED [ 70%]
```

```

tests/test_preprocess.py::TestLoadData::test_no_missing_values PASSED [ 75%]
tests/test_preprocess.py::TestLoadData::test_expected_columns_present PASSED
[ 79%]
tests/test_preprocess.py::TestLoadData::test_target_is_binary PASSED [ 83%]
tests/test_preprocess.py::TestLoadData::test_non_empty PASSED [ 87%]
tests/test_preprocess.py::TestLoadData::test_feature_count PASSED [ 91%]
tests/test_preprocess.py::TestLoadData::test_invalid_path_raises PASSED [ 95%]
tests/test_preprocess.py::TestLoadData::test_handles_missing_values PASSED [1
00%]

```

===== 24 passed, 55 warnings in 1.74s =====

7.2 GitHub Actions CI/CD Pipeline

A 4-stage automated pipeline (`.github/workflows/ci.yml`) runs on every push and PR:

Stage	Tool	Action	Gate
1. Lint	Flake8	Code quality check (--max-line-length=120)	Fail on violations
2. Test	Pytest	Run 24 unit tests (-v --tb=short)	Fail on any test failure
3. Train	Python	Train models, upload heart_model.pkl + MLflow logs	Fail on training error
4. Docker	Docker	Build image, smoke-test /health and /predict	Fail on unhealthy container

8. Model Containerization & API Design

8.1 Docker Configuration

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt
COPY src/ src/
COPY data/ data/
ENV MODEL_PATH=/app/src/models/heart_model.pkl
ENV PYTHONUNBUFFERED=1
EXPOSE 8000
CMD ["uvicorn", "src.api.model_app:app", "--host", "0.0.0.0", "--port", "8000"]

```

Property	Value
Base image	python:3.11-slim (minimal footprint)

Property	Value
Final image size	~450 MB
Exposed port	8000
Health check	/health endpoint

8.2 FastAPI REST API

The prediction API (`src/api/model_app.py`) provides:

Endpoint	Method	Description
/	GET	API information and available endpoints
/health	GET	Health check with model load status
/predict	POST	Heart disease prediction (JSON input)
/metrics	GET	Prometheus-format metrics
/docs	GET	Interactive Swagger UI

Input validation: Pydantic models with field constraints (min/max ranges for all 13 features).

Sample Response:

```
{
  "prediction": 0,
  "prediction_label": "No Heart Disease",
  "confidence": 0.1651,
  "risk_level": "Low Risk",
  "timestamp": "2026-05-06T17:43:36.519802"
}
```

8.3 Flask Browser UI

A user-friendly web interface (`src/api/app.py`) at `http://127.0.0.1:5001` provides: - Grouped form inputs (Demographics, Clinical Measurements, Cardiac Tests) - Real-time prediction with confidence and risk level display - Responsive design that fits within a single viewport

8.4 Docker Build & Run Verification

```
(.venv) mammagan@MAMMAGAN-M-TC73 MLOPs-Final % docker build -t heart-disease-api .
```

```
[+] Building 64.4s (12/12) FINISHED
```

```
(.venv) mammagan@MAMMAGAN-M-TC73 MLOPs-Final % curl http://localhost:8000/health
```

```
{"status":"healthy","pipeline_loaded":true,"timestamp":"2026-05-06T17:43:36.494749"}
```

```
(.venv) mammagan@MAMMAGAN-M-TC73 MLOPs-Final % curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age":55,"sex":1,"cp":2,"trestbps":130,"chol":250,...}'
{"prediction":0,"prediction_label":"No Heart Disease","confidence":0.1651,
"risk_level":"Low Risk","timestamp":"2026-05-06T17:43:36.519802"}
```

9. Production Deployment

9.1 Docker Compose (Full Stack)

The complete application stack is deployed using `docker-compose.yml`:

Service	URL	Description
Heart Disease API	http://localhost:8000	FastAPI prediction server
Prometheus	http://localhost:9090	Metrics collection & alerting
Grafana	http://localhost:3000	Dashboards (admin / admin)

```
docker compose up -d --build
```

Verified deployment (from screenshots/deployment_docker_compose.txt): - All 3 services running and healthy - Prometheus target status: "health": "up" - API prediction working correctly

9.2 Kubernetes Deployment (Minikube)

Production-grade Kubernetes manifests with enterprise patterns:

Manifest	Purpose
k8s/deployment.yaml	2 replicas, rolling update, liveness/readiness probes, resource limits
k8s/service.yaml	LoadBalancer service (port 80 → 8000)
k8s/prometheus-config.yaml	Scrape configuration for K8s service
k8s/prometheus-deployment.yaml	Prometheus Deployment + NodePort Service
k8s/grafana-deployment.yaml	Grafana + auto-provisioned datasource & dashboard

Deployment steps:

```

minikube start --driver=docker
docker build -t heart-disease-api:latest .
minikube image load heart-disease-api:latest
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl rollout status deployment/heart-disease-api --timeout=90s

```

Verified K8s deployment output:

```
(.venv) mammagan@MAMMAGAN-M-TC73 MLOPs-Final % kubectl get pods,svc,deployment
```

NAME	READY	STATUS	RESTARTS	AGE
pod/heart-disease-api-7b5bc5b856-rrsf8	1/1	Running	0	32s
pod/heart-disease-api-7b5bc5b856-tzhgz	1/1	Running	0	32s

NAME PORT(S)	TYPE	CLUSTER-IP	EXTERNAL-IP
service/heart-disease-api-service 80:30578/TCP	LoadBalancer	10.99.80.113	<pending>

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/heart-disease-api	2/2	2	2	32s

Endpoint verification via Minikube tunnel:

```
(.venv) mammagan@MAMMAGAN-M-TC73 MLOPs-Final % curl http://127.0.0.1:62541/health
{"status":"healthy","pipeline_loaded":true,"timestamp":"2026-05-06T18:03:29.385534"}
```

```
(.venv) mammagan@MAMMAGAN-M-TC73 MLOPs-Final % curl -X POST http://127.0.0.1:62541/predict ...
{"prediction":0,"prediction_label":"No Heart Disease","confidence":0.1651,"risk_level":"Low Risk","timestamp":"2026-05-06T18:04:11.280176"}
```

9.3 Kubernetes Deployment Specifications

Feature	Configuration
Replicas	2
Strategy	RollingUpdate (25% max unavailable, 25% max surge)
Liveness probe	HTTP GET /health (delay 10s, period 30s)
Readiness probe	HTTP GET /health (delay 5s, period 10s)
CPU request / limit	250m / 500m
Memory request / limit	256Mi / 512Mi
Image pull policy	Never (local image)

10. Monitoring & Logging

10.1 Application Logging

Structured logging is implemented at the API level (`src/api/model_app.py`):

Aspect	Implementation
Output	Console (stdout) + file (<code>api.log</code>)
Format	<code>%(asctime)s - %(name)s - %(levelname)s - %(message)s</code>
Prediction logs	Input data, result, confidence, latency
Request logs	Timestamp, method, endpoint, status code

Sample log entry:

```
2026-05-06 17:43:36,519 - src.api.model_app - INFO - Prediction: No Heart Dis  
ease |  
Confidence: 0.1651 | Risk: Low Risk | Latency: 0.0103s
```

10.2 Prometheus Metrics

Three production metrics exposed at `/metrics`:

Metric	Type	Labels	Purpose
<code>predictions_total</code>	Counter	<code>result</code>	Track prediction volume by outcome
<code>prediction_latency_seconds</code>	Histogram	—	Monitor inference performance
<code>http_requests_total</code>	Counter	<code>method</code> , <code>endpoint</code> , <code>status</code>	Track API usage patterns

Scrape configuration:

Environment	Target	Interval
Docker Compose	<code>heart-disease-api:8000</code>	15s
Kubernetes	<code>heart-disease-api-service:80</code>	15s

10.3 Grafana Dashboard

A pre-provisioned dashboard **“Heart Disease API Monitoring”** is auto-loaded with:

Panel	Visualization
Total Predictions	Stat (counter)
Predictions by Result	Time series (disease vs healthy)
Prediction Latency (p95)	Gauge

Panel	Visualization
HTTP Request Rate	Time series (by method/endpoint/status)
Average Latency Over Time	Time series

Access: - Docker Compose: `http://localhost:3000` (admin / admin) - Kubernetes: `minikube service grafana-service` (admin / admin)

10.4 Kubernetes Monitoring Stack

Deployed alongside the API on Minikube:

```
kubectl apply -f k8s/prometheus-config.yaml
kubectl apply -f k8s/prometheus-deployment.yaml
kubectl apply -f k8s/grafana-deployment.yaml
```

Service	Access Method	Port
Prometheus	<code>minikube service prometheus-service</code>	9090
Grafana	<code>minikube service grafana-service</code>	3000

Grafana is pre-configured with: - Datasource: Prometheus at `http://prometheus-service:9090` - Dashboard: Auto-provisioned from ConfigMap - Sign-up: Disabled

11. System Architecture

Layer 1: ML Pipeline

Stage	Component	Output
Data Ingestion	<code>data/heart.csv</code>	Raw dataset (303 records)
Preprocessing	<code>src/data/preprocess.py</code>	Cleaned, imputed data
Training	<code>src/models/train.py</code> + MLflow	Experiment logs + metrics
Model Output	GridSearchCV best estimator	<code>heart_model.pkl</code>

Layer 2: Serving

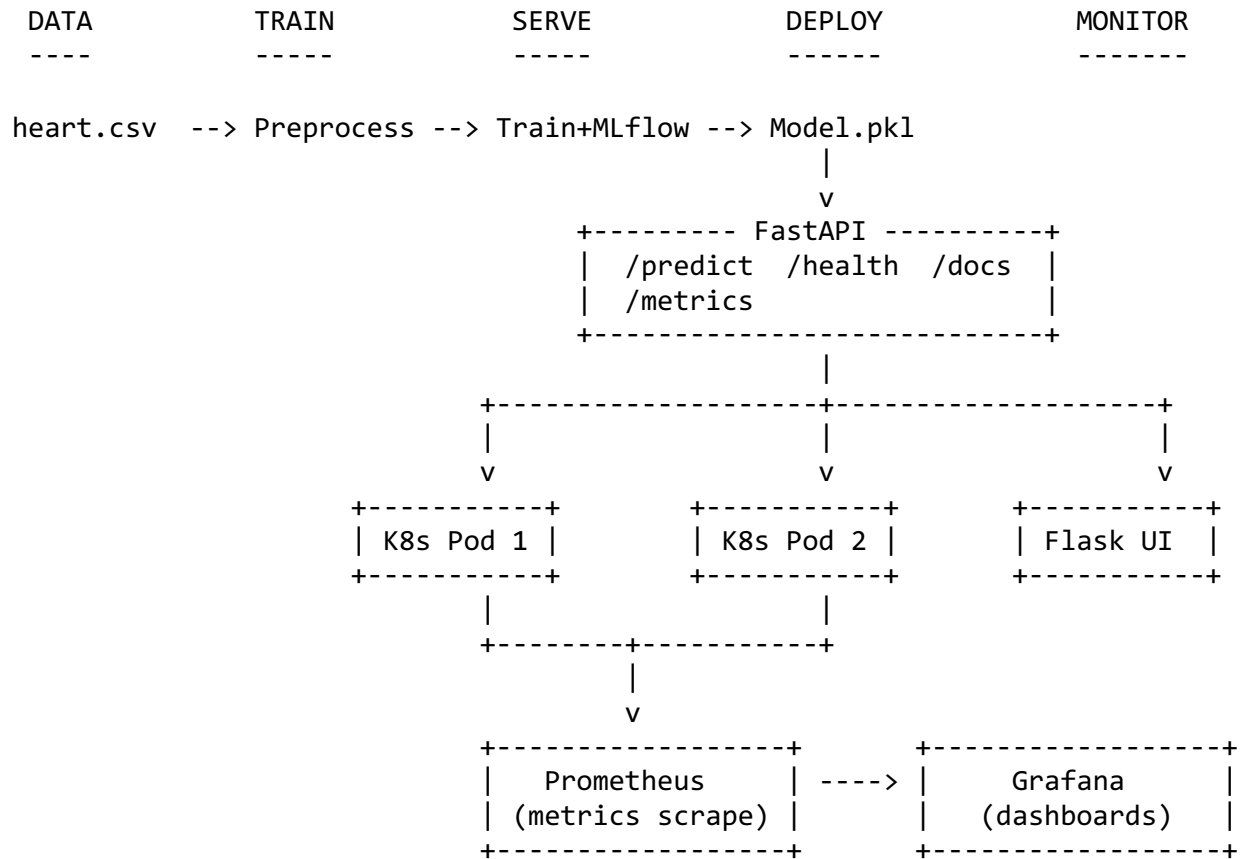
Component	URL	Purpose
FastAPI (REST API)	<code>http://localhost:8000</code>	<code>/predict</code> , <code>/health</code> , <code>/metrics</code> , <code>/docs</code>
Flask UI (Browser)	<code>http://localhost:5001</code>	Interactive prediction form

Layer 3: Deployment & Orchestration

Component	Configuration
Docker Container	<code>python:3.11-slim</code> , port 8000

Component	Configuration
Kubernetes (Minikube)	2 API replicas + LoadBalancer
Prometheus	Scrapes /metrics every 15s
Grafana	Auto-provisioned dashboard

Architecture Flow Diagram



Pipeline Flow:

- Data Layer:** data/heart.csv → src/data/preprocess.py (load, clean, impute)
- Training Layer:** src/models/train.py (GridSearchCV + MLflow tracking) → heart_model.pkl
- Inference Layer:** src/models/inference.py (HeartDiseasePredictor class)
- API Layer:** src/api/model_app.py (FastAPI REST) + src/api/app.py (Flask browser UI)
- CI/CD Layer:** .github/workflows/ci.yml (Lint → Test → Train → Docker)
- Deployment Layer:** Docker Compose / Kubernetes (Minikube) with 2 replicas
- Monitoring Layer:** Prometheus metrics → Grafana dashboards + structured application logging

12. Conclusion & Future Work

12.1 Summary of Deliverables

Component	Status	Details
Data Pipeline	✓ Complete	Automated loading, cleaning, imputation
EDA	✓ Complete	15+ visualizations, statistical analysis
Model Development	✓ Complete	2 models, GridSearchCV, 5-fold CV, ROC-AUC 0.90
Experiment Tracking	✓ Complete	MLflow with params, metrics, artifacts
Testing	✓ Complete	24 tests (preprocessing, model, API)
CI/CD	✓ Complete	4-stage GitHub Actions pipeline
Containerization	✓ Complete	Docker + Docker Compose (3 services)
Orchestration	✓ Complete	Kubernetes with 2 replicas, probes, limits
Monitoring	✓ Complete	Prometheus + Grafana (auto-provisioned)
Documentation	✓ Complete	README, Report, Evidence logs

12.2 Challenges Faced & Solutions

Challenge	Solution
Python version compatibility	Used Python 3.11 in Docker/CI while maintaining local compatibility
Train-serve skew prevention	Embedded StandardScaler inside sklearn Pipeline
CI/CD artifact passing	GitHub Actions upload-artifact / download-artifact across jobs
K8s without registry	minikube image load for local image deployment
Monitoring auto-configuration	Grafana ConfigMaps for datasource + dashboard provisioning

12.3 Future Work

- Add Gradient Boosting (XGBoost/LightGBM) for model comparison
- Implement MLflow Model Registry for versioned model promotion
- Add data drift detection (Evidently AI) for production monitoring
- Deploy to cloud (GCP Cloud Run / AWS ECS) with continuous deployment
- Implement A/B testing framework for model comparison in production
- Add SHAP explanations for model interpretability

13. Evidence Logs

All execution evidence is captured in screenshots/ for reproducibility and audit:

Evidence File	Description
model_training.log	Full training output: GridSearchCV, CV metrics, test evaluation, model selection
Inference.log	Inference test: prediction, confidence, risk level
docker_build.log	Docker image build + container run with API logs
docker_status.log	Container health check + prediction verification
k8s_minikube.log	Minikube start, image load, kubectl apply, pod status, endpoint tests
test_results.txt	Pytest output (24/24 passed)
deployment_docker_compose.txt	Docker Compose stack verification
deployment_k8s.txt	Kubernetes deployment description
*.png	EDA plots, confusion matrices, ROC curves, feature importance, UI screenshots

14. References

#	Resource	URL
1	UCI Heart Disease Dataset	https://archive.ics.uci.edu/ml/datasets/heart+Disease
2	Scikit-learn	https://scikit-learn.org/stable/

#	Resource	URL
3	MLflow	https://mlflow.org/docs/latest/index.html
4	FastAPI	https://fastapi.tiangolo.com/
5	Docker	https://docs.docker.com/
6	Kubernetes	https://kubernetes.io/docs/
7	GitHub Actions	https://docs.github.com/en/actions
8	Prometheus	https://github.com/prometheus/client_python
9	Grafana	https://grafana.com/docs/
10	Project Repository	https://github.com/Matha-51/mlops-assignment

Report generated: May 2026